



Memory Management(1)

Dept. of Computer Science
Hanyang University

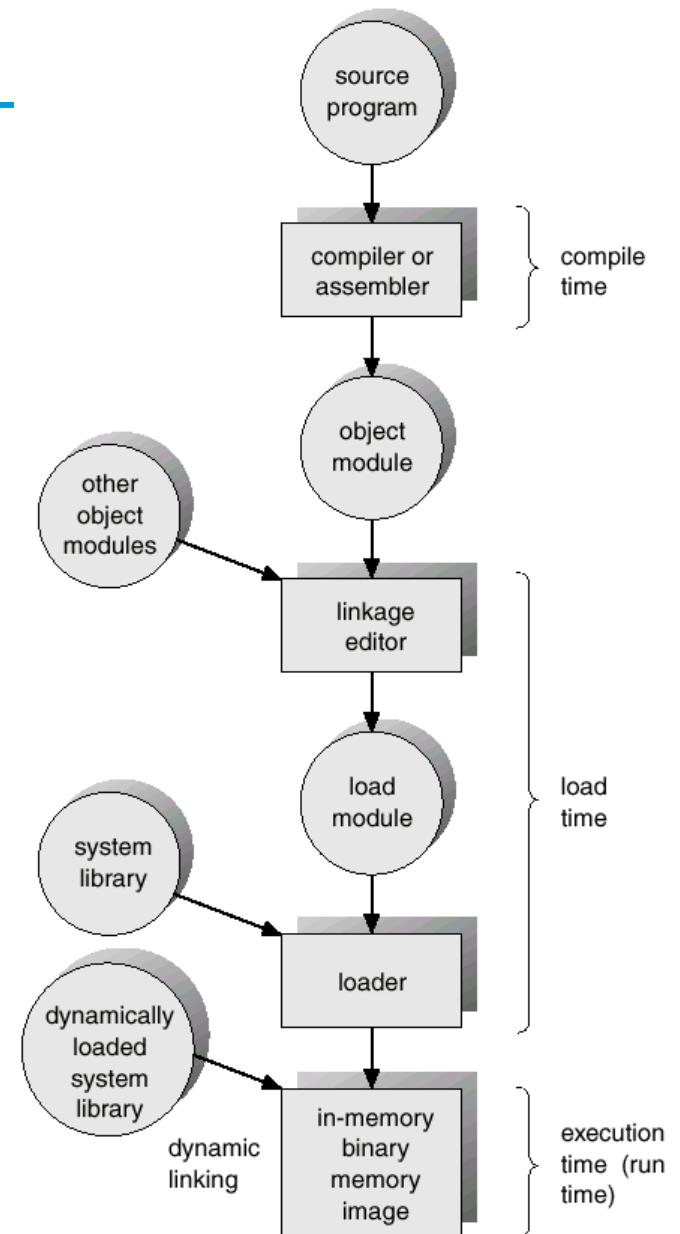




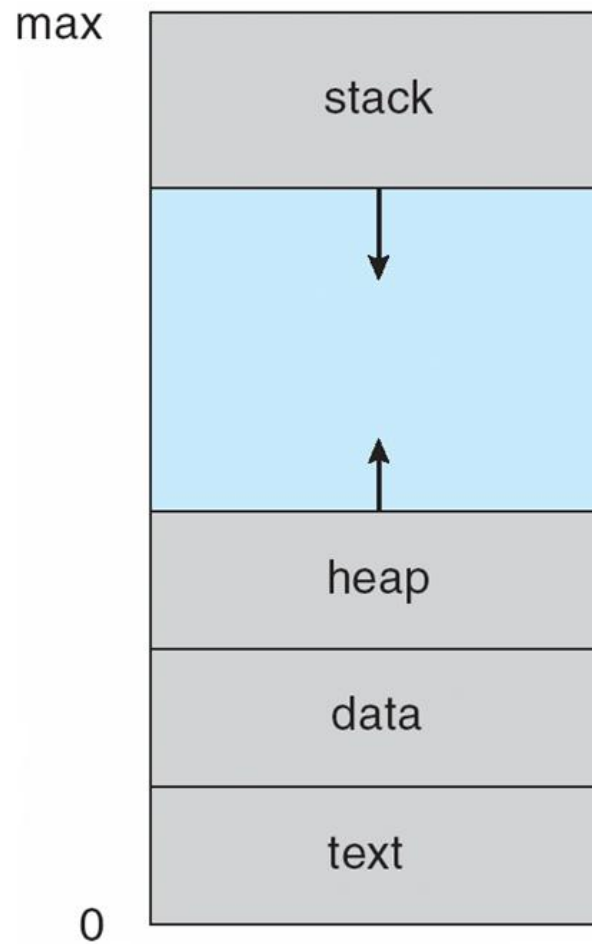
- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are the only storage CPU can access directly
- Register access in one CPU clock cycle (or less)
- Main memory can take many cycles
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

Multistep Processing of a User Program

User programs go through several steps before being run on memory.



Process in Memory: Address Space





Binding of Instructions and Data to Memory

- **Compile time binding**

- Absolute address of each symbol must be known at this time
- absolute code containing absolute address is generated
- must recompile code if starting location changes

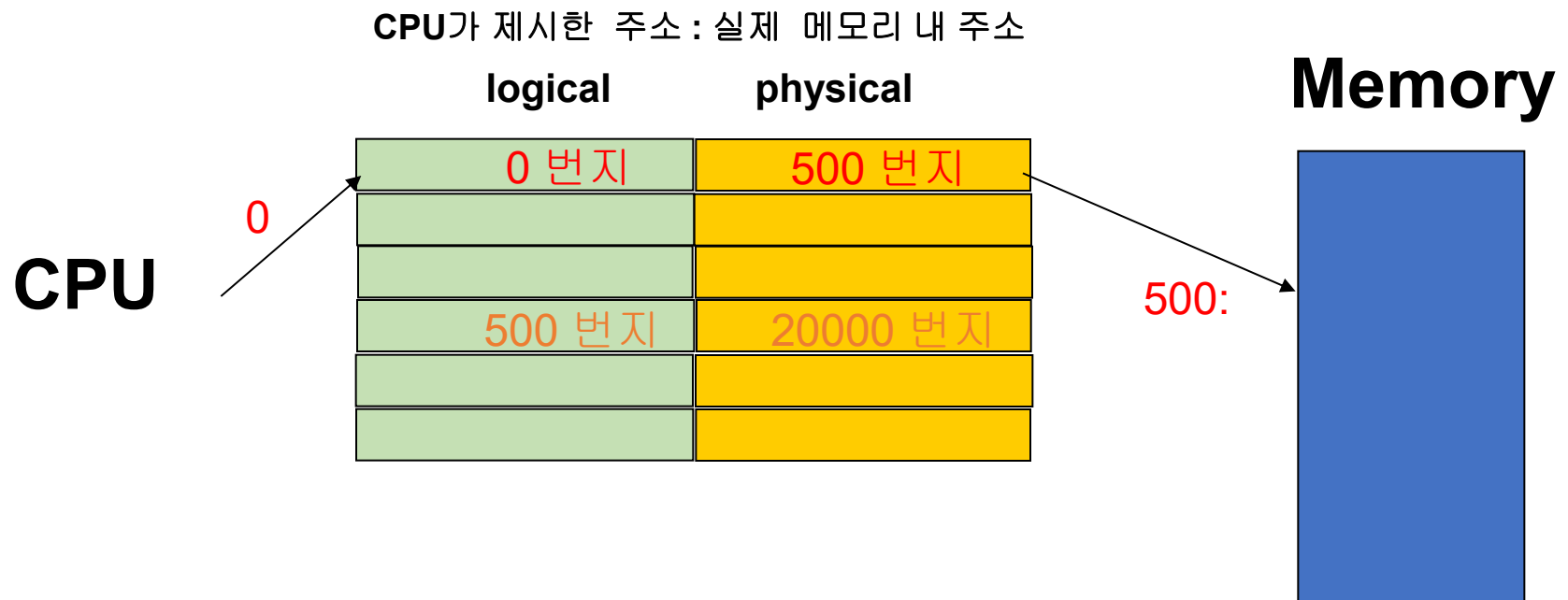
- **Load time binding**

- Loader assigns absolute address to each symbol
- Compiler generates relocatable code containing relative addresses

- **Execution time binding**

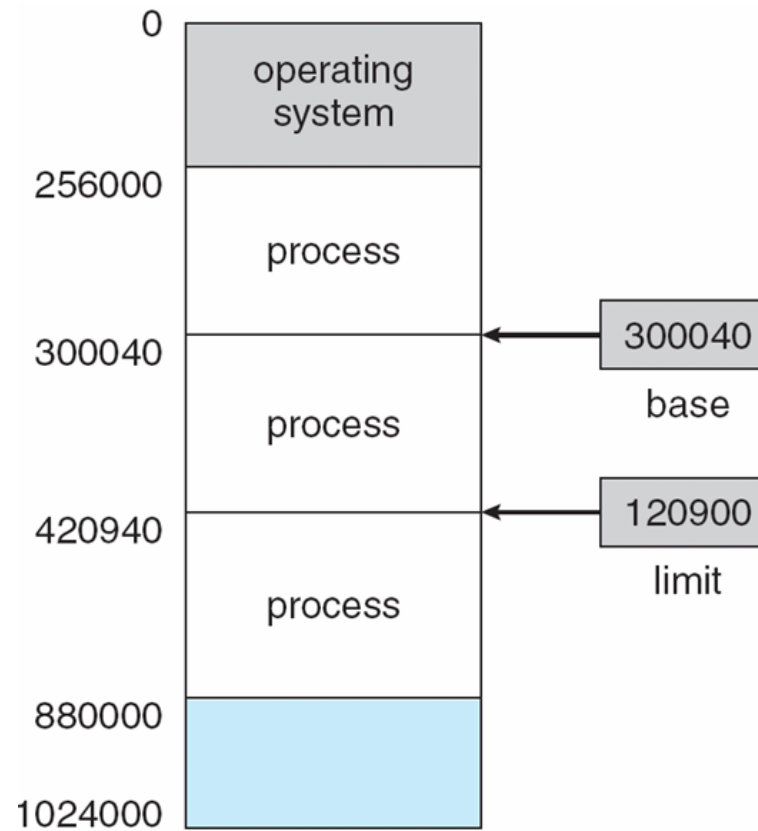
- Used when process moves its in-memory location during execution
- Whenever CPU generates address, binding is required (address mapping table)
- need hardware support (e.g., *base and limit registers, MMU*)

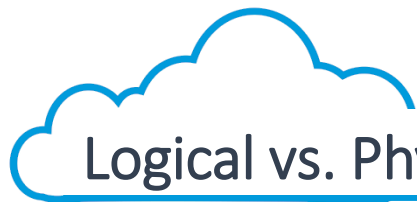
Address Mapping Table



Base and Limit Registers

- A pair of **base** and **limit** registers define the physical address space





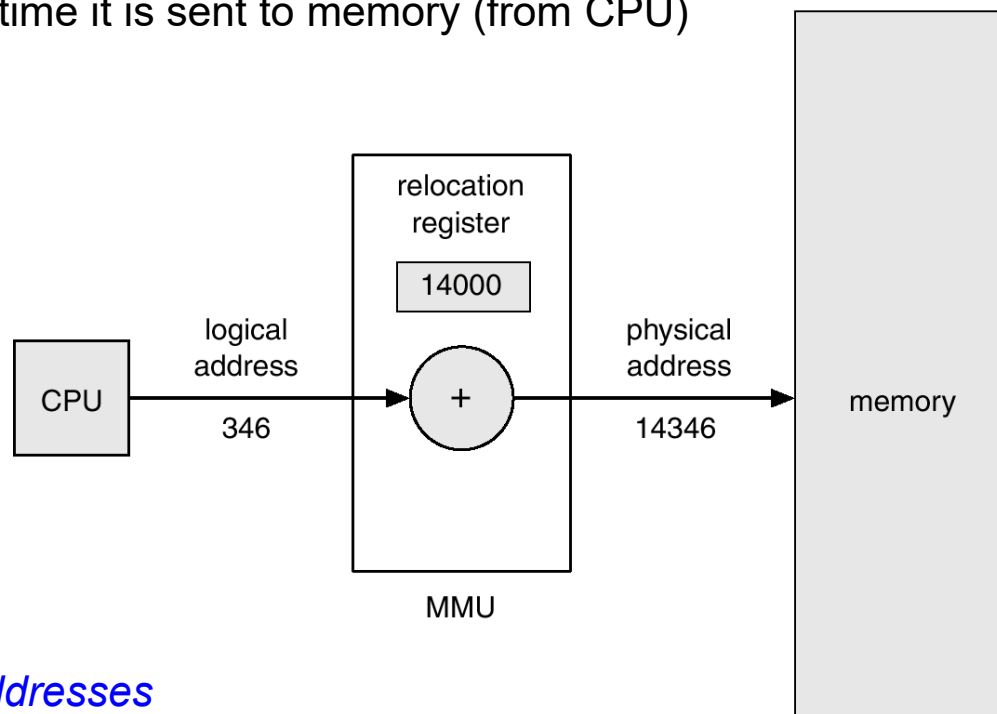
Logical vs. Physical Address Space

- The concept of a **logical address space** that is bound to a separate **physical address space** is central to proper memory management
 - **Logical address**
 - Generated by the CPU
 - Also referred to as **virtual address**
 - **Physical address**
 - Address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes
- Logical (virtual) and physical addresses differ in execution-time address-binding scheme



Memory-Management Unit (MMU)

- **MMU (Memory-Management Unit)**
 - A *Hardware device* that maps *virtual address* to *physical address*
- In MMU scheme
 - the value in the *relocation register* is added to every address generated by a user process at the time it is sent to memory (from CPU)



- The user program
 - deals with *logical addresses*
 - *never sees* the *real physical addresses*



Swapping necessitates dynamic relocation

- **Swapping**

- A process can be *swapped* temporarily out of memory to a *backing store* and then brought back into memory sometime later

- **Backing store**

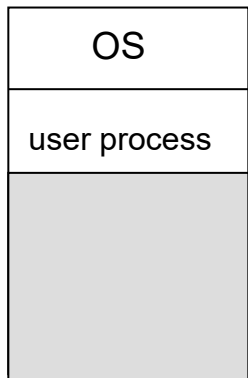
- Fast disk large enough to accommodate all memory images for all users
- Must provide direct access to these memory images

- Major part of swap time is transfer time

- Total transfer time is proportional to the amount of memory swapped

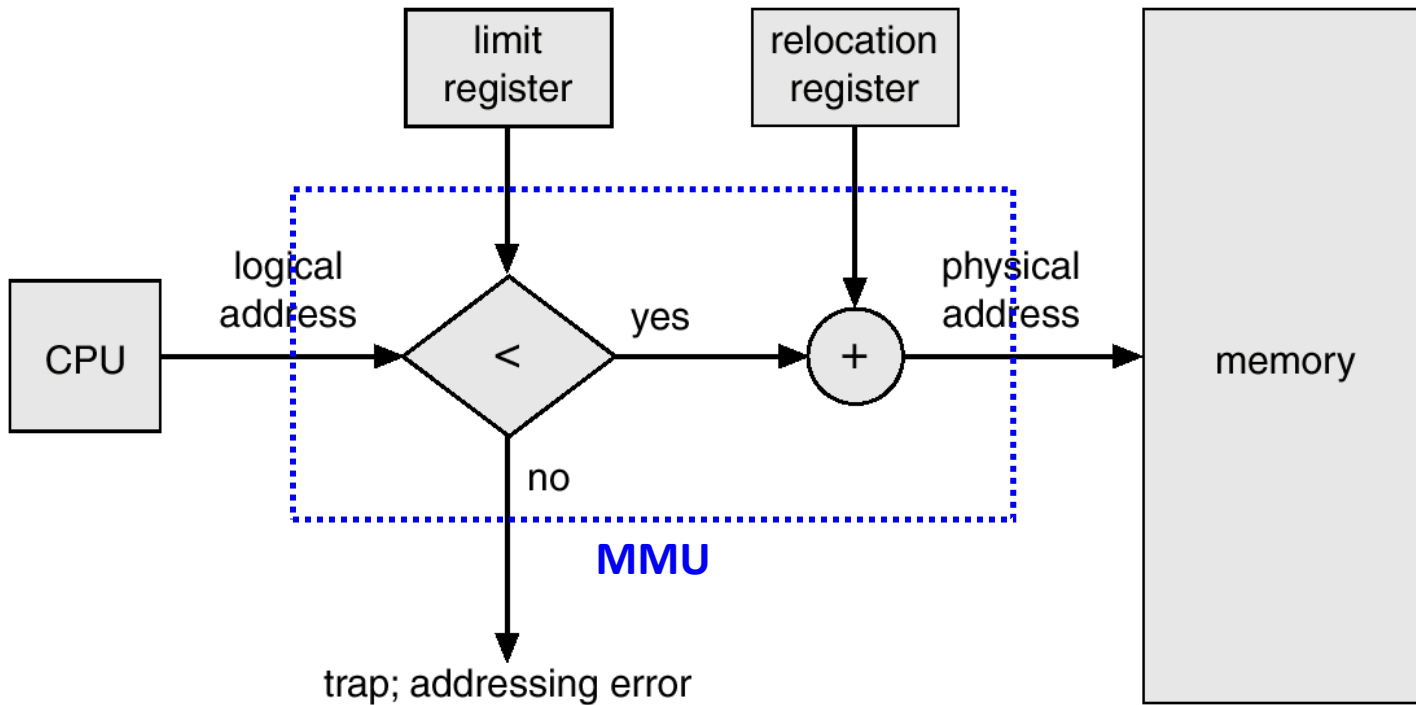
Contiguous Allocation

- Main memory is usually divided into two partitions:
 - *Resident operating system*, usually held in *low memory* with interrupt vector
 - *User processes* then held in *high memory*
 - To *protect* user processes from each other and OS



- **Relocation register**
 - contains value of the smallest physical address
- **Limit register**
 - contains range of the logical addresses
 - each logical address bounded by limit register

Hardware Support for Relocation and Limit Registers





Contiguous Allocation (Cont.)

- **Hole** : block of available memory
 - Holes of various size are scattered throughout memory
 - When a process arrives, it is allocated memory from a hole large enough to accommodate it
 - Operating system maintains information about:
 - a) allocated spaces b) free spaces (hole)

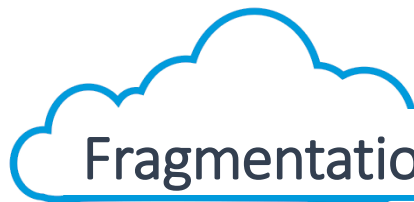


Dynamic Storage-Allocation Problem

How to satisfy a request of size n from a list of free holes

- **First-fit**
 - Allocate the *first hole* that is big enough
- **Best-fit**
 - Allocate the *smallest hole* that is big enough
 - must search entire list, unless ordered by size
 - Produces many small leftover hole
- **Worst-fit**
 - Allocate the *largest hole*
 - must also search entire list
 - Produces the largest leftover hole

First-fit and best-fit better than worst-fit in terms of storage utilization



Fragmentation

- **External fragmentation**
 - Total memory space exists to satisfy a request, but it is not contiguous
- **Internal fragmentation**
 - Allocated memory may be slightly larger than requested memory; this size difference is internal fragmentation : **memory internal to a partition, but not being used**
- Reduce external fragmentation by **compaction**
 - Shuffle memory contents to place all free memory together in one large block
 - Compaction is possible only if relocation is dynamic, and is done at execution time

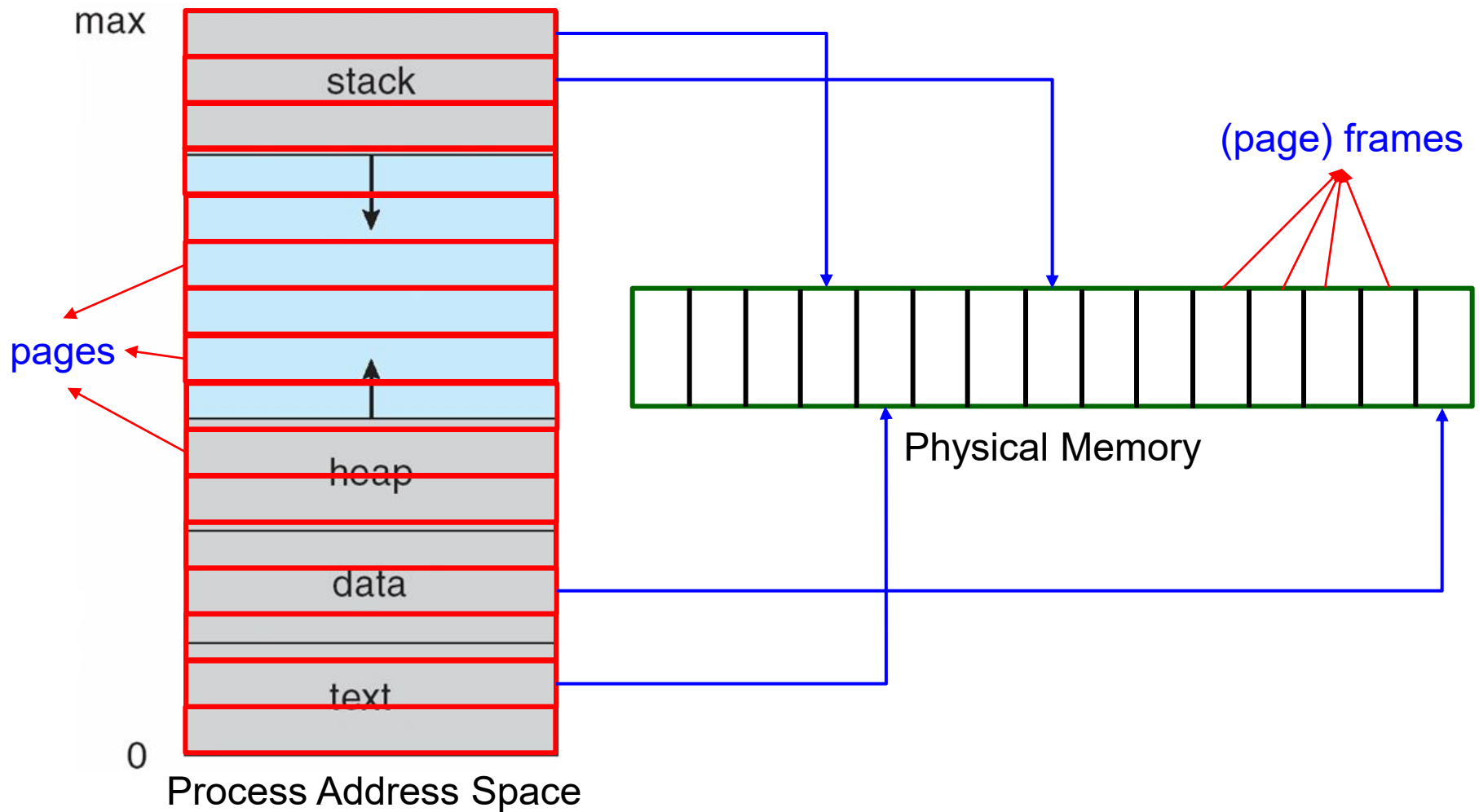


- **Paging**

- is a scheme that *permits address space* to be *noncontiguous*

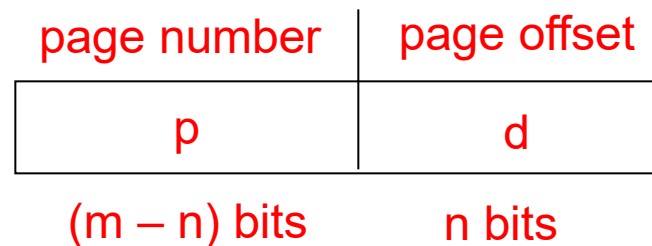
- Basic Method

- Divide physical memory into *fixed-sized blocks* called *frames* (size is power of 2, between 512 bytes and 8 MB)
- Divide logical memory into *blocks of same size* called *pages*
- Keep track of all free frames
- To run a program of size *n* pages, need to find *n* free frames and load program
- Set up a *page table* to translate logical to physical addresses
- *Internal fragmentation, but no external fragmentation*



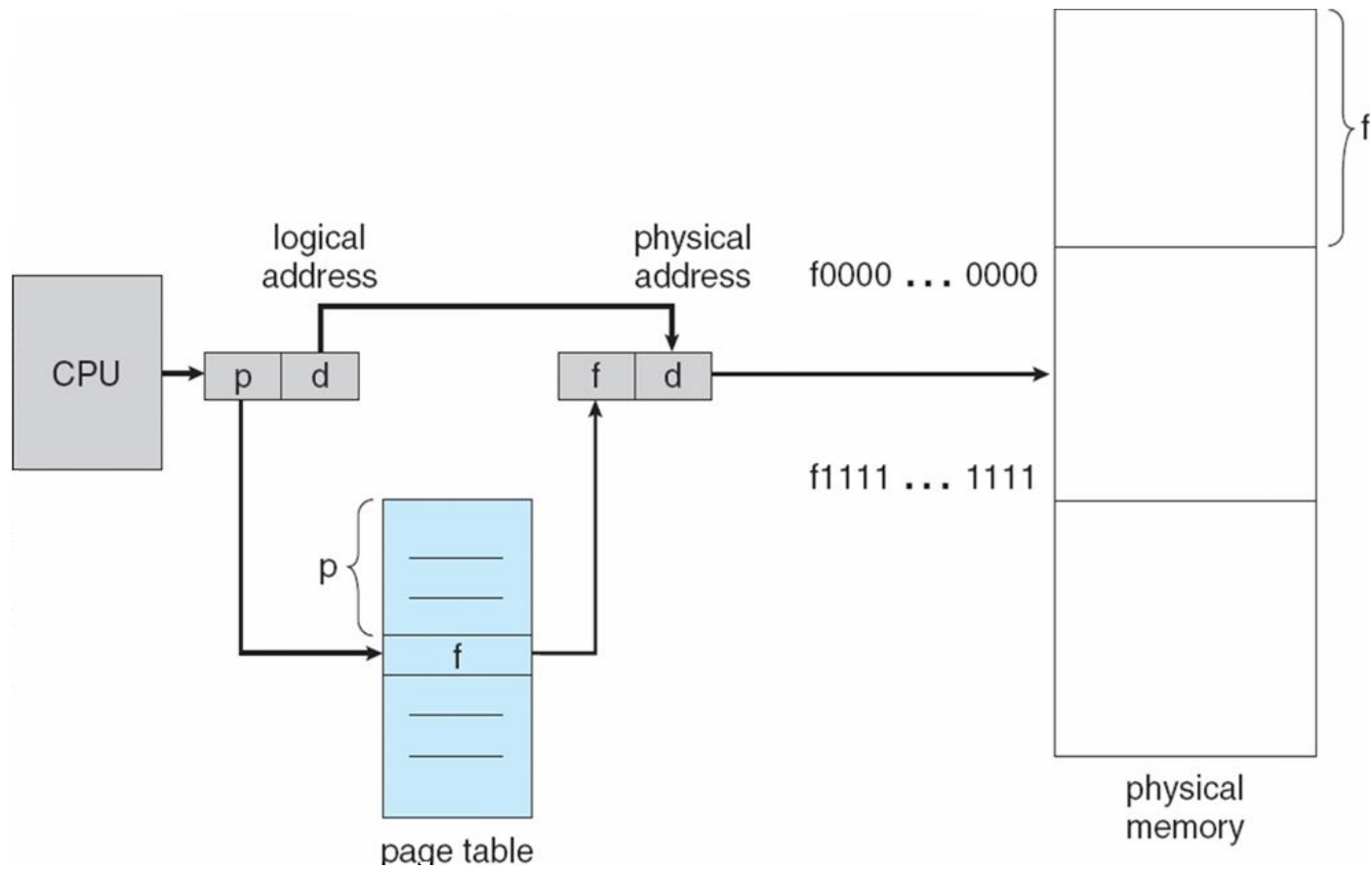
Address Translation Scheme in Paging

- Address generated by CPU is divided into:
 - **Page number (p)** – used as an index into a *page table* which contains base address of each page in physical memory
 - **Page offset (d)** – combined with base address to define the physical memory address that is sent to the memory unit

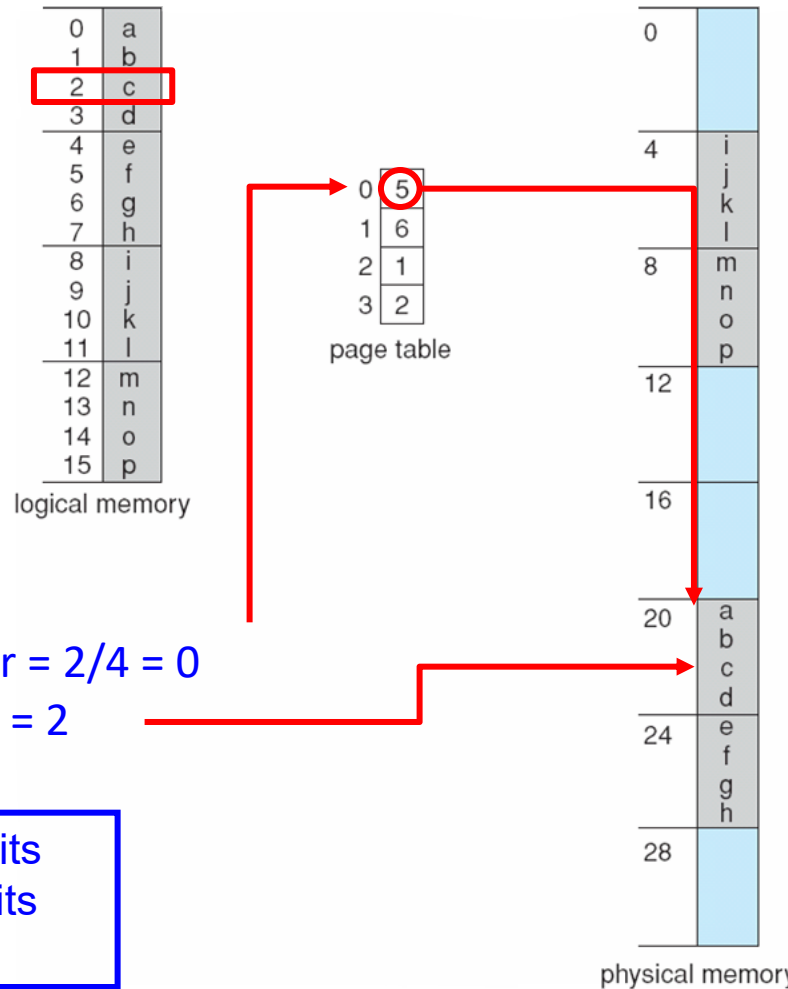


- For given logical address space 2^m and page size 2^n

Address Translation Architecture



Paging Example



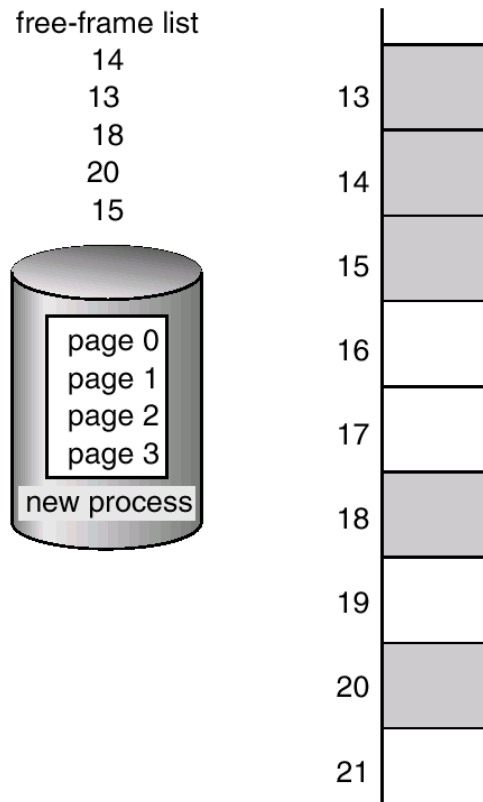
Page Number = $2/4 = 0$

Offset = $2\%4 = 2$

Logical address (m): 4 bits
Page number (m-n): 2 bits
Offset (n): 2 bits

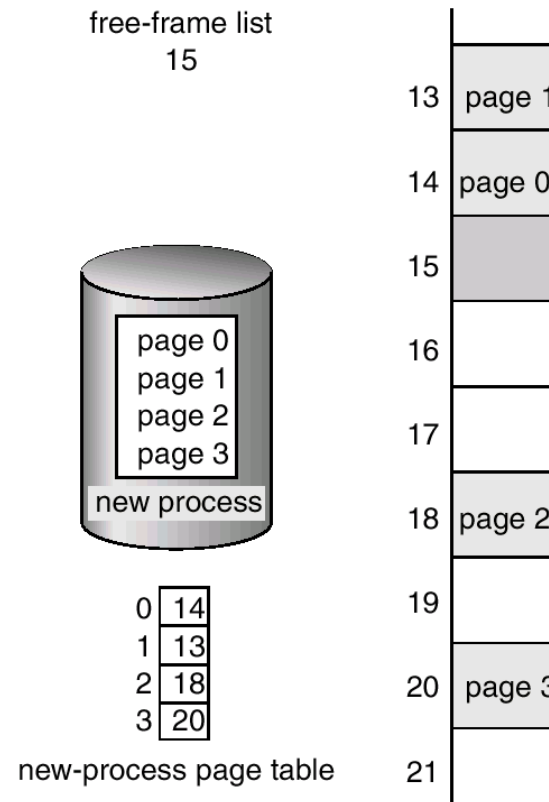
4-byte page $n = 2$ bits

Free Frames



(a)

Before allocation



(b)

After allocation



Implementation of Page Table

- Page table is *kept in main memory*
 - *Page-table base register (PTBR)* points to the *page table*
 - *Page-table length register (PTLR)* indicates *size of the page table*
- **Problem:** *Every memory (data/instruction) access requires two memory accesses*
 - One for the *page table* and one for the *data/instruction*
- **Solution:** A special fast-lookup hardware cache called *Translation Look-aside Buffer (TLB)* or *Associative memory* is used
 - In general, *TLB is flushed (remove old entries) at context switching time*
 - Some TLBs store address-space identifiers (ASIDs) in each TLB entry, to avoid frequent TLB flushing
 - Uniquely identifies each process to provide address-space protection for that process

Associative Memory (TLB)

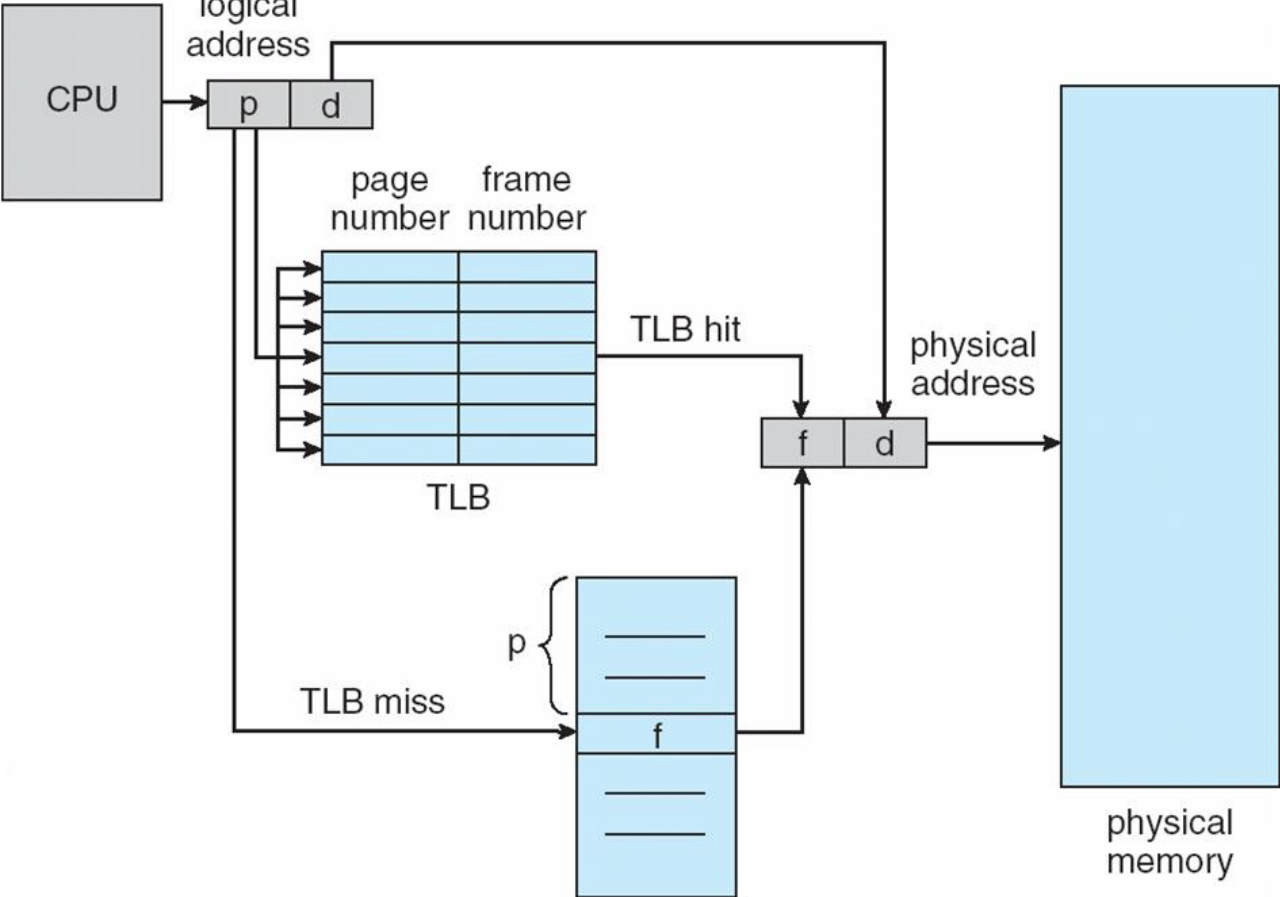
- Two types of memory
 - General memory: (ex: DRAM)
 - give address -- return record
 - Associative memory: (ex: Phone book)
 - give field of record -- return record
 - Very slow without parallel search (or indexing)
 - High cost for implementation

- Associative Memory** (TLB) – parallel search

- Since only part of page table is in TLB record

Page #	Frame #

- Address translation (p, d)
 - Try portion of page table in associative memory first
 - If p is in associative register, get frame # out
 - Otherwise, get frame # from page table in main memory





Effective Access Time

- Associative lookup = α time unit
- memory access time = β
- **Hit ratio** = ε (percentage found in the associative memory)
- Effective Access Time (EAT)

$$\begin{aligned} & \begin{array}{cc} <\text{hit}> & <\text{miss}> \end{array} \\ \text{EAT} &= (\alpha + \beta) \varepsilon + (\alpha + 2 \beta)(1 - \varepsilon) \\ &= \alpha + (2 - \varepsilon) \beta \end{aligned}$$



Memory Protection

- Memory protection implemented by associating protection bit with each frame
- *Valid-invalid bit* - each entry in the page table:
 - “*valid*” indicates the page is legal (a valid page)
 - The associated page is in the process’ logical-address space
 - “*invalid*” indicates otherwise (access not allowed)

Valid (v) or Invalid (i) Bit in a Page Table

- Page size = 2 KB
- Uses only addresses 0 to 10468
- 6 ($=10469/2048$) pages are allocated to the process
- Only 6 entries are used in the page table
- PTLR can be used to test the validity of the address instead of the valid-invalid bit

00000	page 0
	page 1
	page 2
	page 3
	page 4
10,468	page 5
12,287	

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page n



Shared Pages

- *Shared code*
 - One copy of read-only (*re-entrant*) code shared among processes (eg, text editors, compilers, window systems)
 - Shared code must appear in same location in the logical address space of all processes
 - To enable self-reference in the shared code
- *Private code and data*
 - Each process keeps a *separate copy* of the data
 - The pages for the private code and data can appear anywhere in the logical address space

Shared Pages Example

Editor is shared (Editor consists of 3 pages -- ed1, ed2, ed3)

